

Asymmetric Cryptography

Part 2

Computer Systems Security

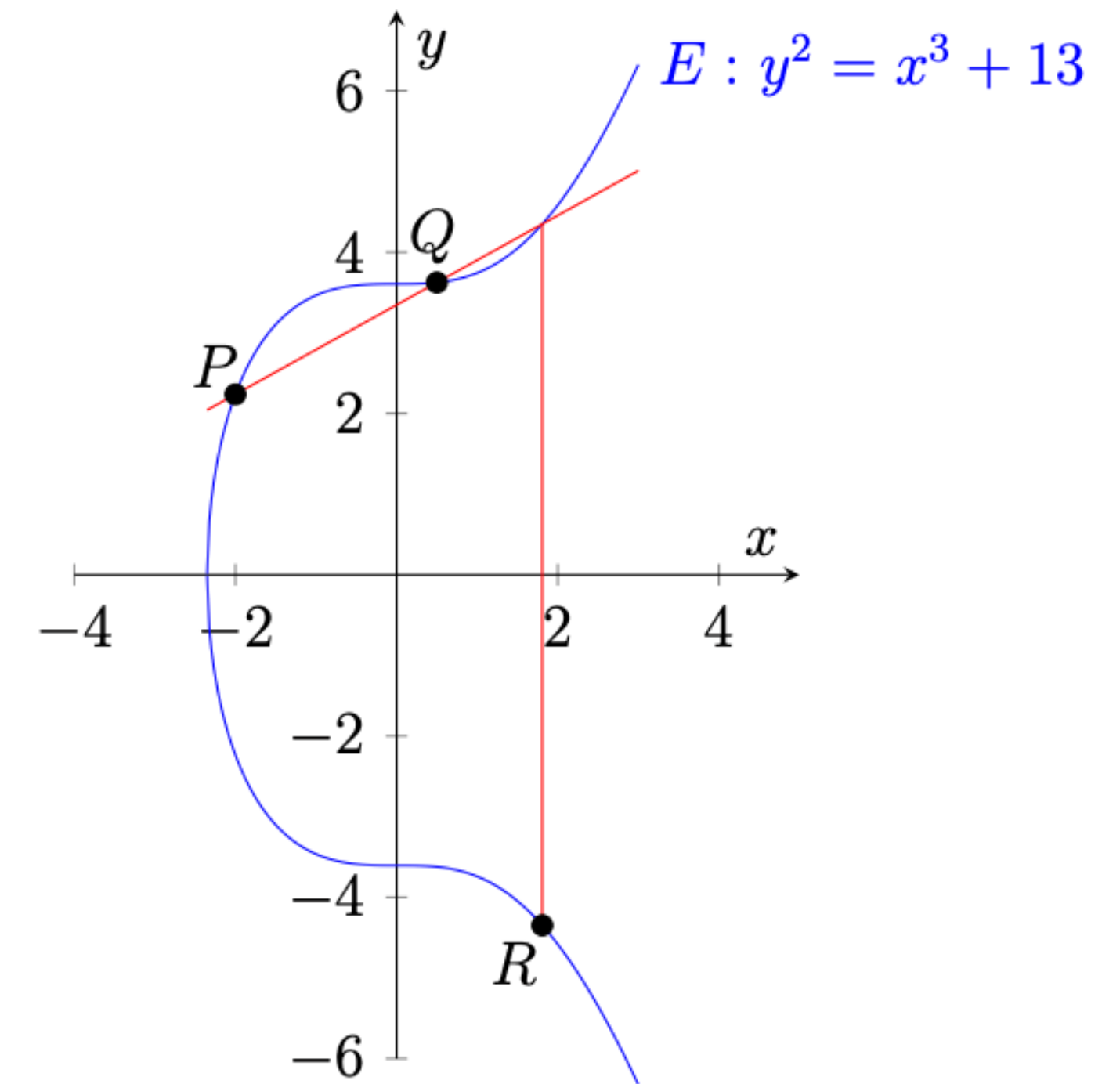
2025/26

LEI - UM

Hugo Pacheco hpacheco@di.uminho.pt

Elliptic-Curve Cryptography

- The discrete logarithm problem can be expressed on any finite structure such as $GF(p)$
- But it can also be "transposed" to a cyclic group defined on an elliptic curve
- In Elliptic-Curve Cryptography (ECC)
 - Easy: modular exponentiation $\Rightarrow$ scalar multiplication
 - Hard (under certain criteria): discrete log $\Rightarrow$ Elliptic Curve Discrete Logarithm
- Variants of the primitives based on the discrete logarithm can be defined to operate on elliptic curves (e.g. ECDH, ECDSA, etc)



Elliptic-Curve Cryptography

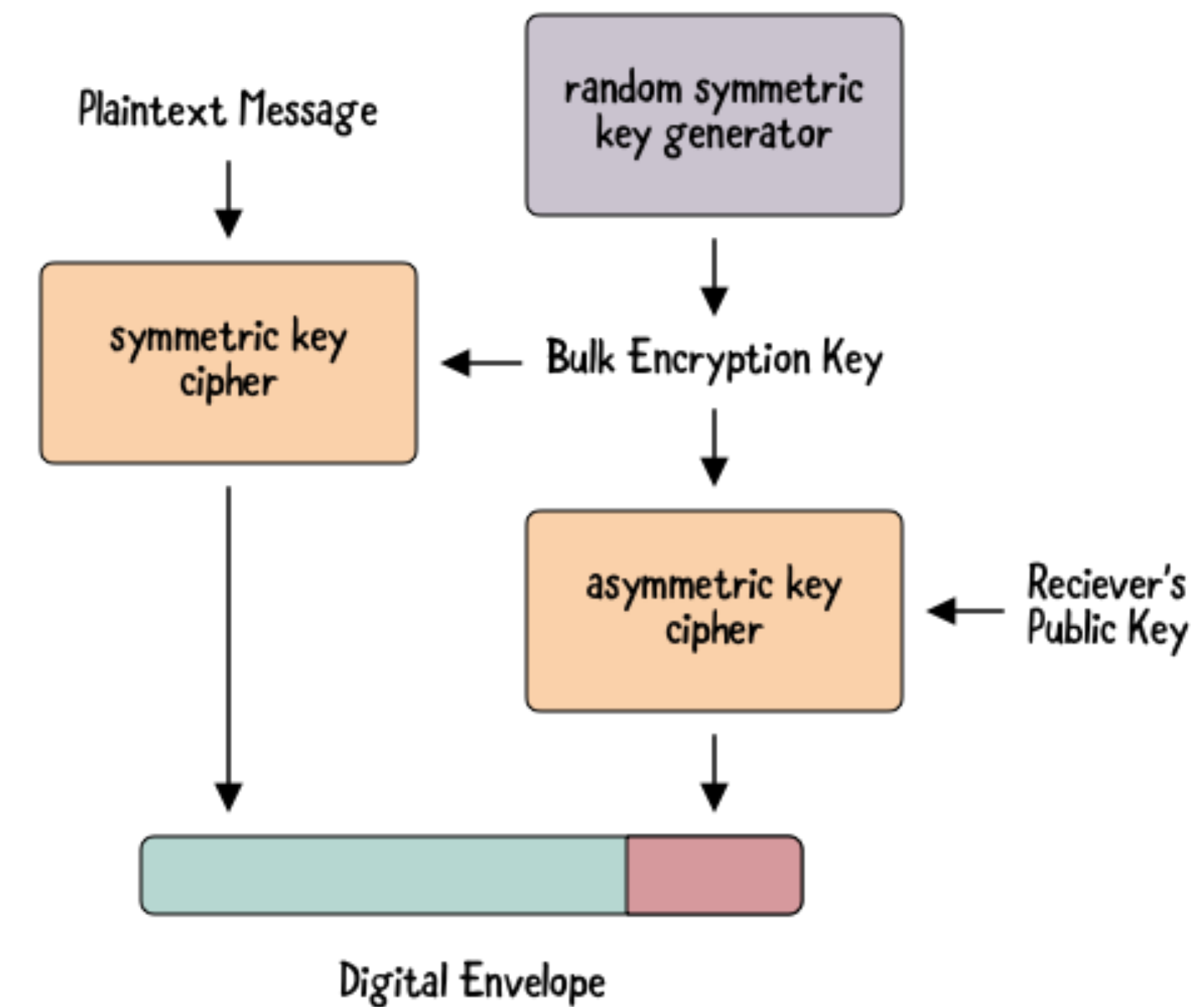
Security

- The great advantage of ECC is that it allows much smaller keys for the same security level
- A critical aspect is the choice of parameters
 - The nature of the criteria and the computational effort typically involved mean that the only viable way is to use standardised parameters
- Possible choices:
 - NIST-P (e.g. P-256, P384, P-521): proposed by NSA
 - Curve 25519: parameters proposed by D. Bernstein

NIST guidelines for public key sizes for AES			
ECC KEY SIZE (Bits)	RSA KEY SIZE (Bits)	KEY SIZE RATIO	AES KEY SIZE (Bits)
163	1024	1 : 6	
256	3072	1 : 12	128
384	7680	1 : 20	192
512	15 360	1 : 30	256

Hybrid Encryption

- A PKE scheme is only able to handle messages of a restricted shape
- But unlike block ciphers, where modes of operation lift encryption to arbitrary messages, efficiency considerations lead to **Hybrid Encryption**
 - Symmetric cipher to encrypt the actual message
 - Symmetric PKE scheme to encrypt a randomly generated symmetric key
- In its simpler form, it takes the structure of what is known as a **digital envelope**



Hybrid Encryption

KEM/DEM

- Hybrid encryption typically follows the KEM/DEM paradigm
 - **Key Encapsulation Mechanism (KEM)**: a public-key scheme that is tailored to generate (symmetric) secret keys and their encapsulation by PKE

KeyGen : SecParam $\rightarrow$ (SecKey, PubKey)

Encap: PubKey $\rightarrow$ (SymKey, CipherText)

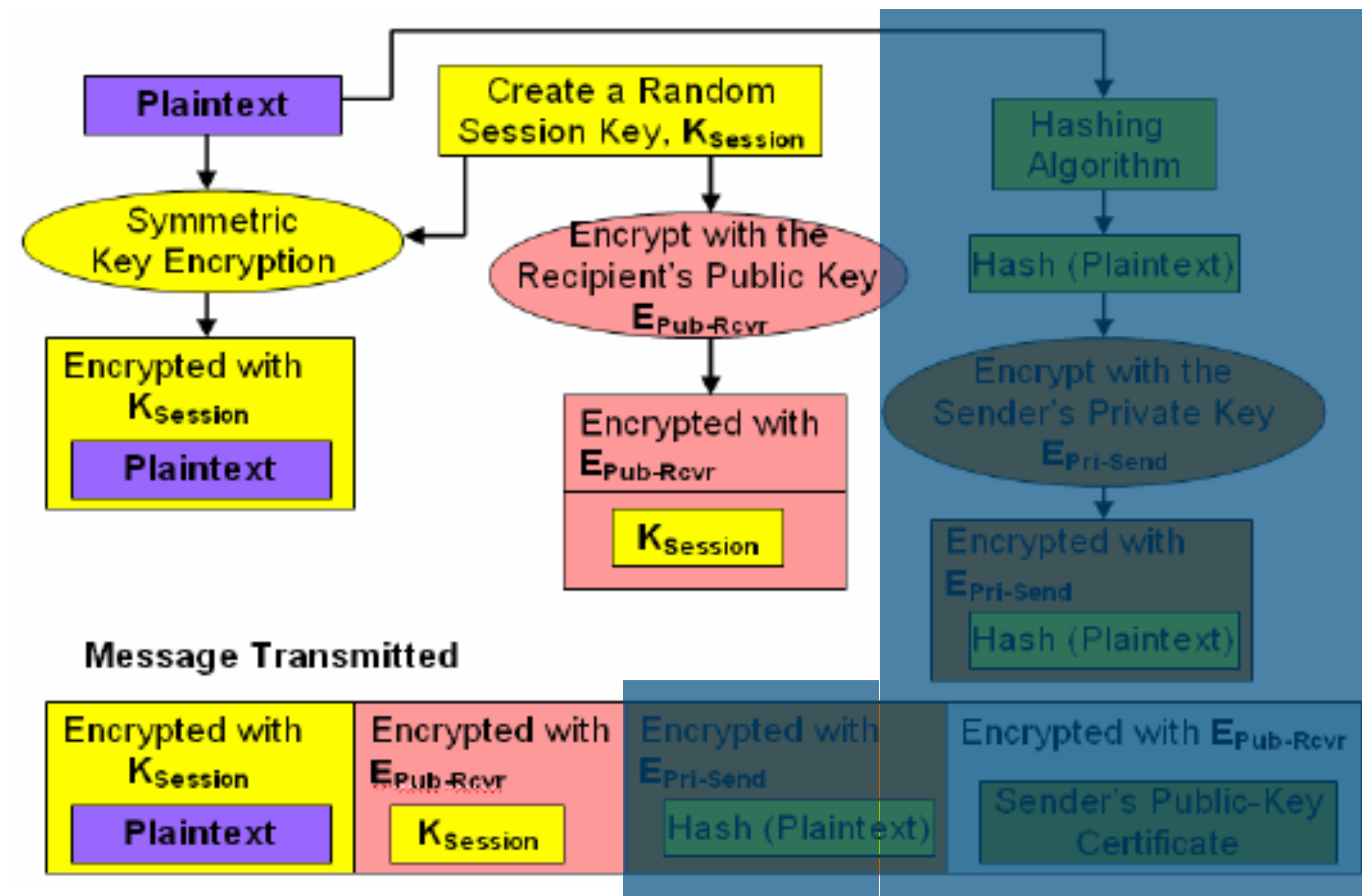
Decap: (SecKey, Ciphertext) $\rightarrow$ SymKey

- **Data Encapsulation Mechanism (DEM)**: a symmetric scheme that uses the KEM-generated key (e.g. an authenticated encryption scheme)

Hybrid Encryption

KEM/DEM

- Used in various applications, e.g., TLS, messaging apps, e-mail S/MIME
- **Does not escape the vulnerability of non-authentic public-keys**
 - Asymmetric cryptography always depends on a reliable association between public keys and identities
- How can a receiver check that a public key is authentic?
 - **Digital signatures, certificates $\Rightarrow$ next**



Physical signatures

- Expected properties
 - Assurance of authorship of message/document
 - Document not tampered after signature
 - Non-falsifiable
 - Non-reusable
 - Non-repudiation
- **Realistic Guarantees?**
 - Many, many assumptions...
 - How (easily) can we subvert these guarantees?



Digital Signatures

- Expected properties
 - Assurance of authorship of message/document
 - Document not tampered after signature
 - Non-falsifiable
 - Non-reusable
 - Non-repudiation
- **Strong guarantees?**
 - Also entails assumptions...



Digital Signatures

Concept

- Often recognised as the main contribution of asymmetric cryptography
 - To be able to provide asymmetric authentication guarantees, e.g. when signing documents
- Intuition:
 - Swap roles of public/secret keys in PKE
 - Only **signer** (with secret key) can produce signatures
 - Any **verifier** (with public key) can verify signatures
- Properties:
 - Integrity: the message is not modified after signing
 - Authenticity: the identity of the signer can be confirmed
 - **Non-repudiation**: it is possible to prove the identity of the signer
- Recall that MACs were able to establish the first two properties...
 - ... but — as a symmetric scheme — signer/verifier had the same capabilities

Digital Signatures

Construction

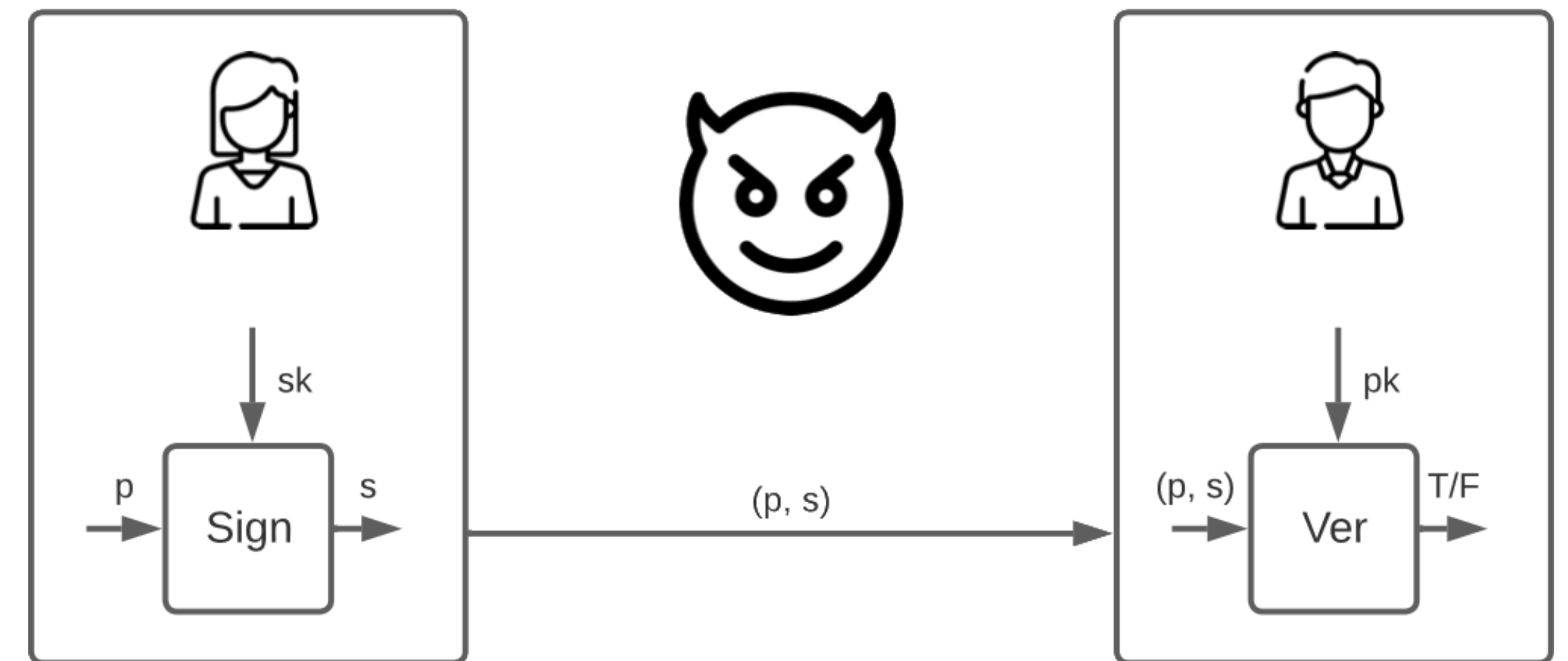
- Two entities: signer and verifier
- A signature scheme comprises three operations:

$KeyGen: SecParam \rightarrow SecretKey * PubKey$

$Sign: SecretKey * Message \rightarrow Signature$

$Ver: PubKey * (Signature * Message) \rightarrow Bool$

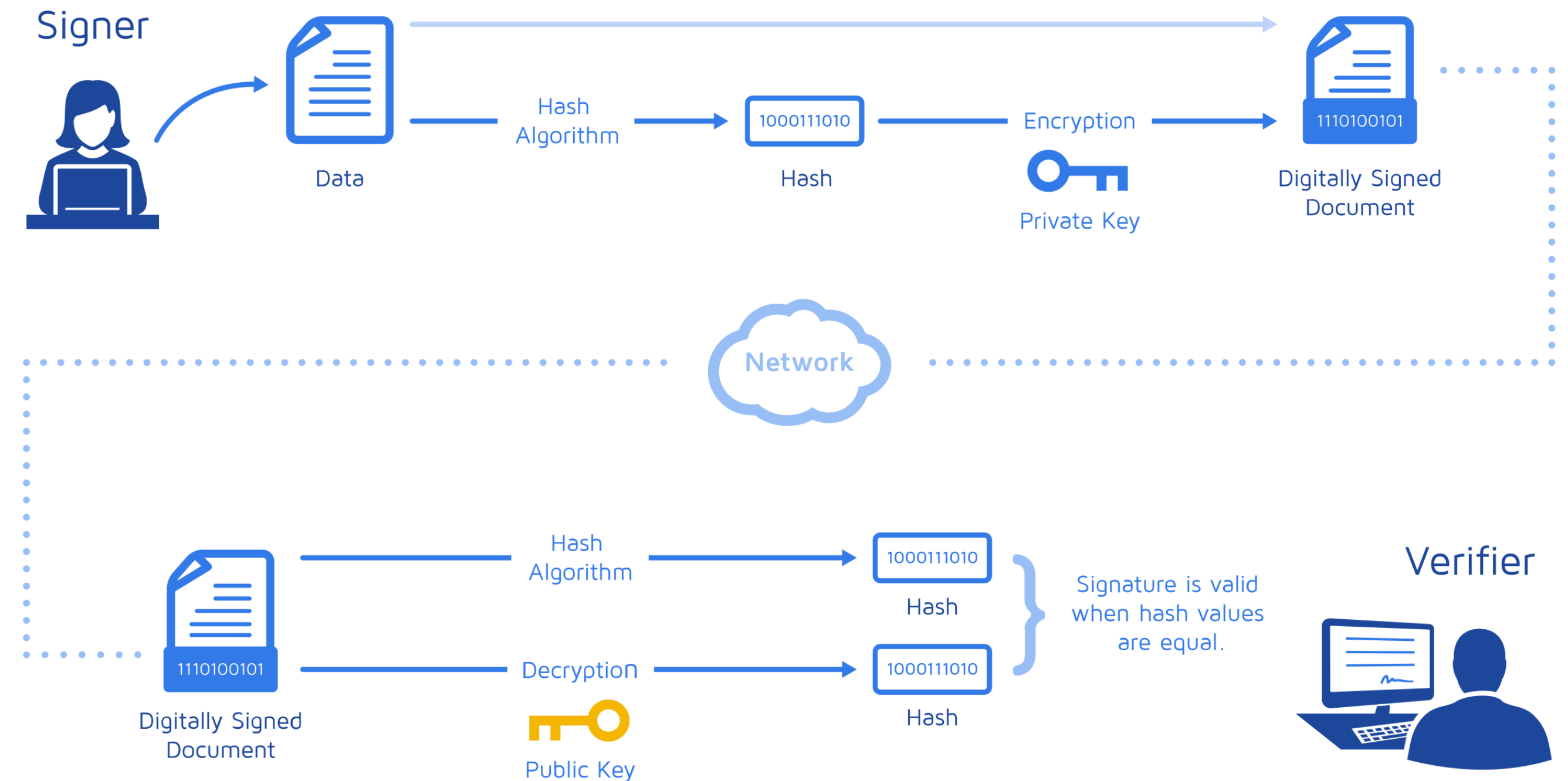
- Properties (for a signer's key-pair (sk, pk)):
 - **Correctness:** $Ver(pk, (Sign(sk, M), M)) = true$
 - **Security:** impossible to forge a signature σ for any message M such that $Ver(pk, (M, \sigma)) = true$ without knowing sk
- Note that the signature σ is attached to the message M , but can be orders of magnitude smaller



Digital Signatures

Hashing

- Cryptographic hash functions are always used together with digital signatures
 1. To map the content of arbitrary messages into small digests
 2. To break patterns (together with a padding scheme) that the mathematical structure of PK algorithms would otherwise expose



Digital Signatures

Security

- Guarantee integrity (signature is valid) and non-repudiation (signee cannot deny having signed the document)
- **But (still) do not guarantee (complete) authenticity**
 - A man-in-the-middle attack may succeed to carry out the verification of the signature with the “wrong” public key
- Notice again a **circularity**:

A digital signature is intended to establish the authenticity of a document, but relies on the authenticity of the association between public-keys and identities

- Trust is needed to establish the association of identities and public-keys
⇒ PKI (next)

Digital Signatures

RSA

- We can reverse RSA to build a digital signature
 - KeyGen (just the same)
 - $\text{Sign}(sk, m) = \text{Dec}(sk, H(m))$ (secret exponent over a message digest)
 - $\text{Ver}(pk, (\sigma, m)) = \text{Enc}(pk, \sigma) \equiv H(m)$ (invert signature and check message digest)
- Correctness (not derived from correctness of PKE):
 - $\text{Ver}(pk, (\text{Sign}(sk, m), m)) \equiv \text{true} \Leftrightarrow \text{Enc}(pk, \text{Dec}(sk, H(m))) = H(m)$
 - Note that encryption/decryption in RSA are commutative: $(x^d)^e = (x^e)^d$
- Note that, without hashing, anyone could forge signature the signature 1^d for message 1
- Standardized as RSA-PSS (Probabilistic Signature Scheme)

Digital Signatures

Digital Signature Algorithm (DSA)

- Developed by the NSA for the NIST in 1991, included in the Digital Signature Standard (DSS)
- Security based on the hardness of the discrete logarithm (a variant of El-Gamal)
 - Like ElGamal, relies crucially on a random “one-time” mask per signature
- When compared to RSA-based signatures:
 - Designed to provide a very efficient signature procedure
 - Requires much less storage space
 - Particularly suited to running in environments with limited resources
 - But much heavier verification phase

Digital Signatures

7th generation game consoles

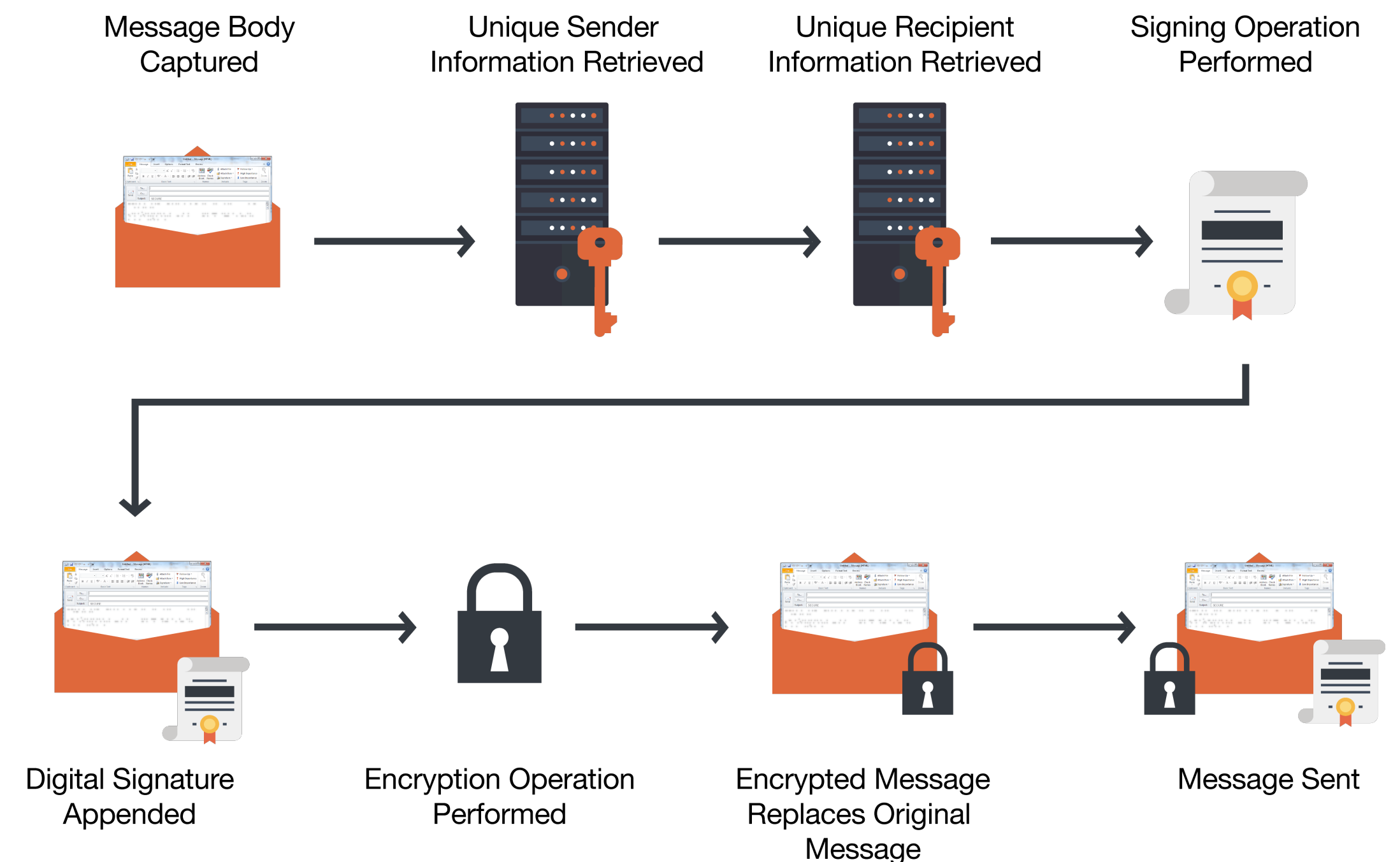
- Digital signatures to enforce official game content (but broken by hacker group Fail0verflow)
 - Sony PS3 + **DSA**
 - **Constant (not random nor one-time) mask for all signatures**
 - Public key hardcoded into the hardware
 - Nintendo Wii + **RSA**
 - **Ignore the padding bytes, consider only hash of the original message**
 - Software strncmp vulnerability



Digital Signatures

S/MIME

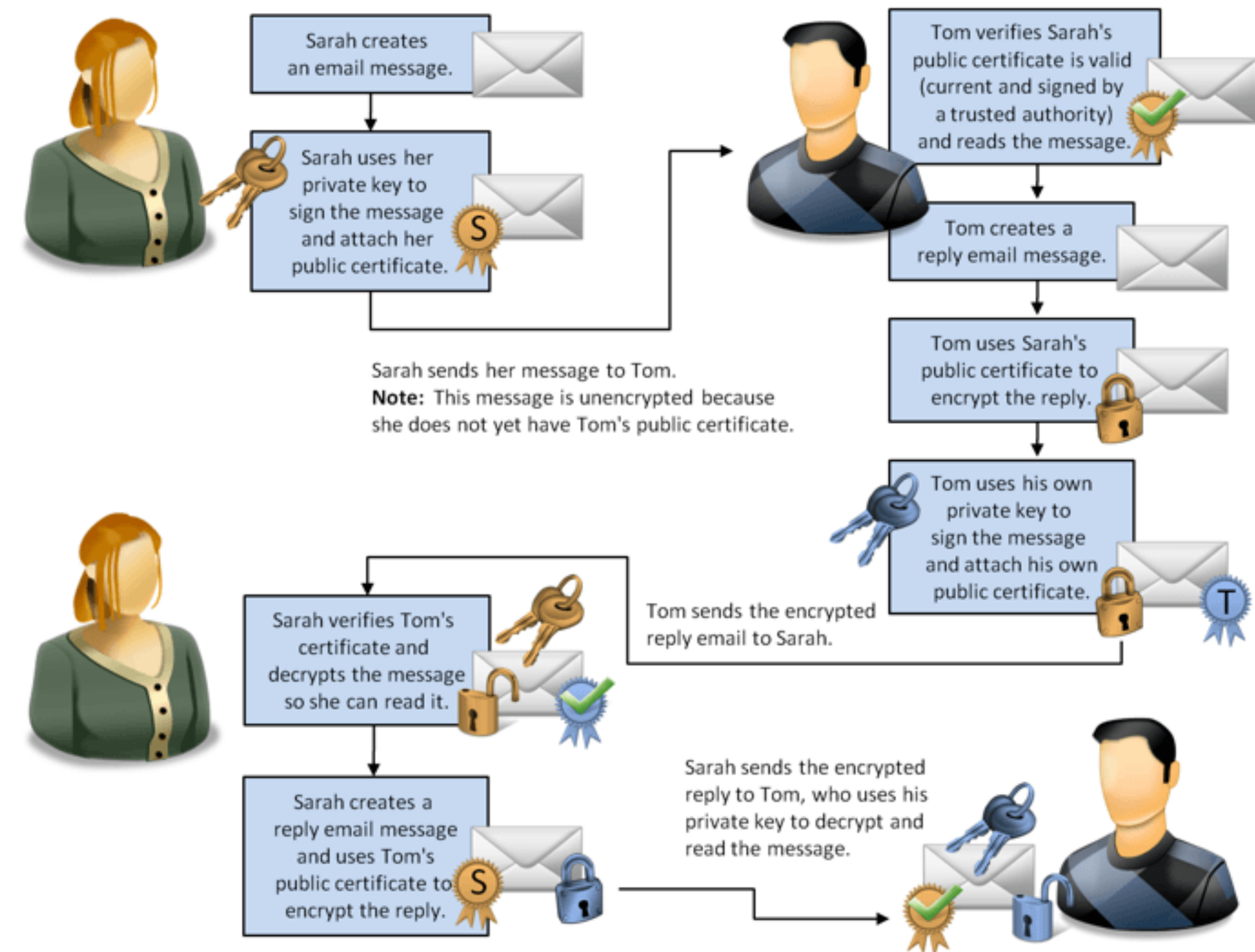
- RFC standard does not determine a specific operation order
- **Sign-then-Encrypt** (preferred by mail clients)
 - Protects sender's information (encrypted)
 - Guarantees non-repudiation (message came from sender) and confidentiality
 - **But it may have been sent to someone else**
 - To mitigate such a forwarding attack, S/MIME mandates to include recipient on the signed message
 - **Can still be forwarded, receiver knows that it was not meant for him**



Digital Signatures

S/MIME

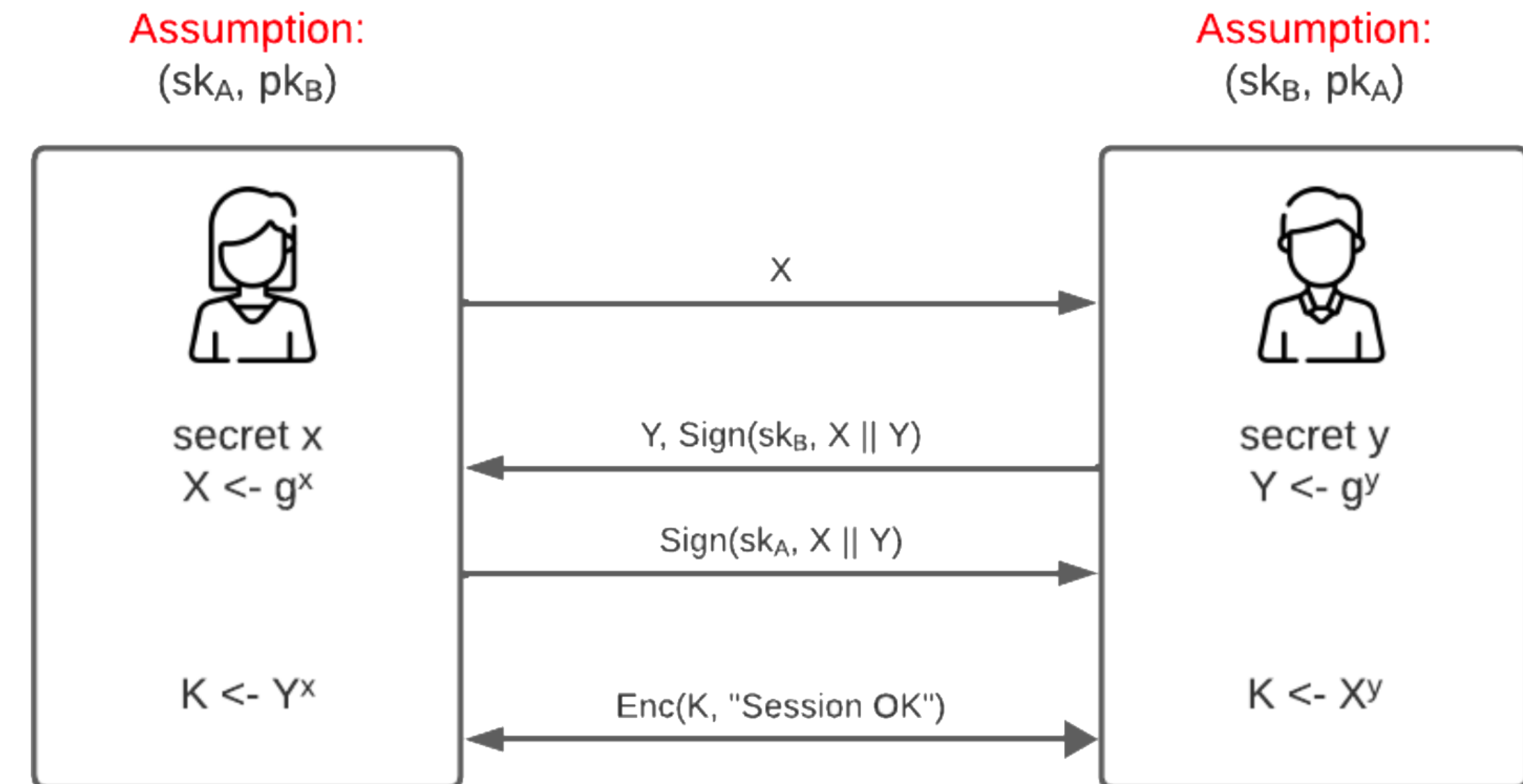
- RFC standard does not determine a specific operation order
- **Encrypt-then-Sign** (preferred by mail gateways)
 - Gateways can validate the signature (e.g. spam filters) without being able to decrypt
 - As before, guarantees message integrity (has not been tampered) and confidentiality
 - **But not non-repudiation**
 - Only that sender signed the ciphertext...
 - Original message has not been signed!



Authenticated Diffie-Hellman

Station-to-station protocol

- To guarantee forward secrecy, (Ephemeral) DH uses fresh key pairs per execution
- But DH is “anonymous” (no authenticity)
⇒ MitM attacks
- The **station-to-station protocol** extends DH with digital signatures
 - Attacker cannot sign on behalf of A or B
 - Why do parties sign $X \parallel Y$ and not K ?
 - K is an “opaque” secret, does not bind X or Y



An Imminent Threat

Quantum computers

- A drastic change in how we do computation
 - Relies on the laws and phenomena of quantum mechanics
 - qubits allow the superposition of information
 - Certain (believed) hard problems can be solved in (believed) much more efficient ways
 - Up to exponential speedup



Quantum Computers

What about cryptography?

- Symmetric cryptography
 - Keys are “random” values
 - Grover’s algorithm for unstructured search
 - Quadratic speedup (checking 2^n keys in $2^{\frac{n}{2}}$)
 - AES keys have to double in size
 - **Not too big a deal**
- Asymmetric cryptography
 - Key pairs have algebraic structure
 - Shor’s algorithm for solving the hidden subgroup problem
 - Polynomial-time solutions for all hard PKE problems
 - PKE is broken
 - **Huge deal**

Quantum Computers

Predictions for the announced fate...

- Not reasonable to suspect quantum will break currently used cryptographic algorithms
 - Today's quantum computers ~1K qubits (IBM)
 - Far below the ~1M qubits necessary to break 2048-bit RSA
 - Probably possible to break around 2035
- So... not a real threat today?
 - **Definitely yes**
 - “Store-now-decrypt-later” a.k.a. “Harvest-now-decrypt-later” a.k.a. “Y2Q”
 - Today, there is certainly the capacity (e.g. governments) to store massive amounts of encrypted communications for highly sensitive information, in the hope of cracking it in the near future

Post-Quantum Cryptography

Rethinking math

- Post-Quantum Cryptography (PQC) refers to classical cryptographic techniques that are secure against quantum adversaries
 - Not to be confused with Quantum Cryptography, on the use of quantum computers for cryptography
 - Why may quantum computers break PK cryptography?
 - Efficiently finding hidden subgroup, whose period / order depends on secret key
 - Ideas for quantum-safe problems?
 - Add noise / error to calculations
 - Massive unstructured search spaces
- Are there problems that are believed to be **hard for quantum computers**?
 - **Lattice-based Cryptography**
 - Learning With Errors (LWE)
 - Shortest Vector Problem (SVP)
 - Closest Vector Problem (CVP)
 - Multi-variable systems of equations
 - Hash-based Cryptography
 - Code-based Cryptography
 - Isogenies in Super-Singular Elliptic Curves
 - ...

Post-Quantum Cryptography

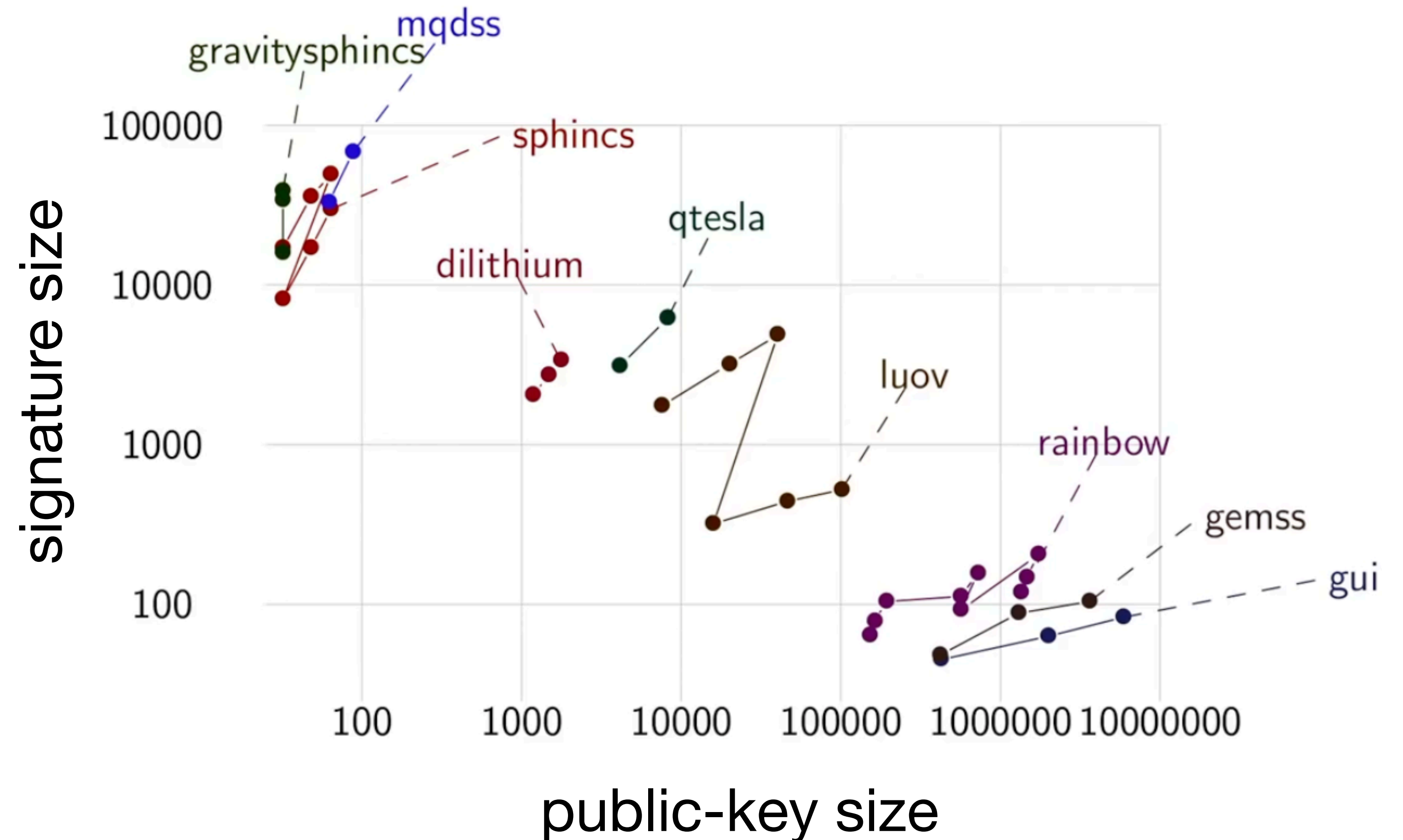
A calculated transition

- There has been a hasty effort to promote the transition to quantum-safe cryptographic algorithms
- NIST initiative for PQC standardisation (finished in 2023)
 - Different requirements from previous calls (e.g., SHA-3):
 - Less confidence on the actual threat of / security against quantum computer
 - Not expected to result in “pick a winner”
 - Ideally, a selection of “good alternatives”
 - Easy drop-in replacements are unlikely
- Current PQC Standards:
 - FIPS 203, Module-**Lattice-Based** Key-Encapsulation Mechanism Standard (ML-KEM) (formerly known as CRYSTALS-Kyber)
 - FIPS 204, Module-**Lattice-Based** Digital Signature Standard (ML-DSA) (formerly known as CRYSTALS-Dilithium)
 - FIPS 205, Stateless Hash-Based Digital Signature Standard (SLH-DSA) (formerly known as SPHINCS+)
 - Other standardisation initiatives are underway...
 - E.g., Falcon (Fast Fourier **Lattice-Based** Compact Signatures over NTRU)

Post-Quantum Cryptography

A wide range of alternatives

- Different compromises
 - Underlying problem
 - Security margin
 - Key sizes
 - Efficiency
 - ...



Post-Quantum Cryptography

Current status

- But new PQC algorithms still lack maturity
- PQC needs time to become a complete replacement
 - More memory / network bandwidth for significantly larger keys
 - Trust (e.g. research in cryptanalysis)
 - New implementations
 - Usability and interoperability (e.g. redesign protocols)
 - Standards development
- On the other hand, the **transition should be as early as possible** so that we are already protected against “Store-now-decrypt-later” attacks

Post-Quantum Cryptography

Hybrid schemes

- **The use of hybrid schemes is recommended at an early stage in the adoption of PQC**
- Combine a classic cryptographic protocol with a PQC one
- Advantages:
 - Interoperability with “legacy” systems
 - Protects the risk associated with the novelty of PQC
 - At least as secure as the best of the combined schemes
- Disadvantages:
 - Still makes problems resulting from size more pronounced (public key, signature, etc)
 - Redundancy (computational overhead, etc)